FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Gabriel A. Cuesta
Section:	AN43
Professor:	Crisola G. Tan
Date:	October 12, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

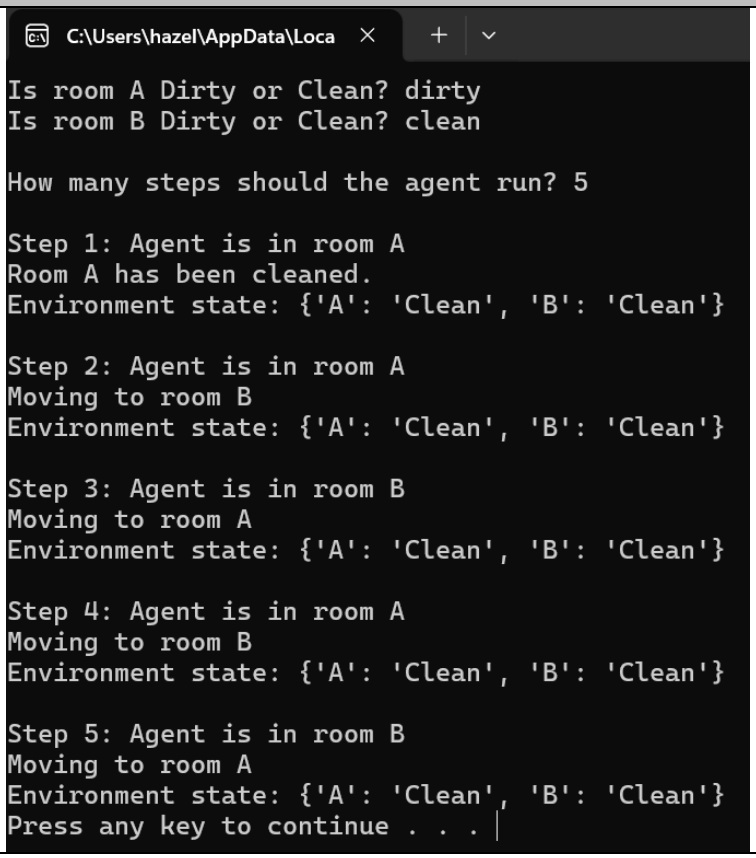
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

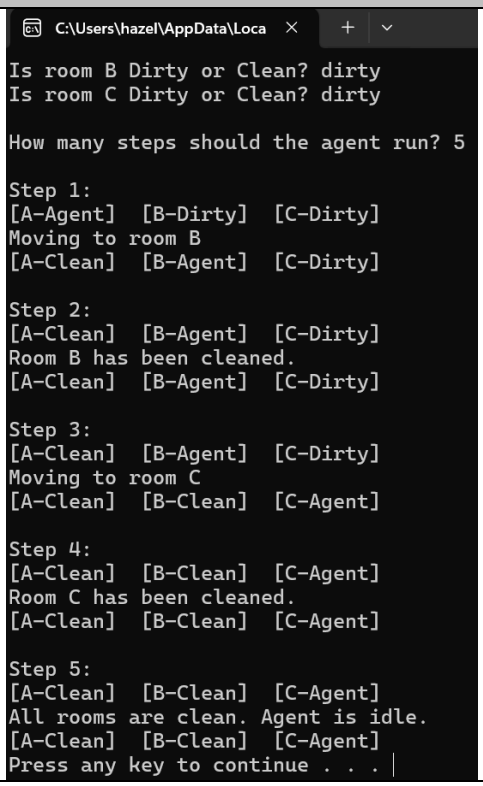
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Code

```
import random

# ENVIRONMENT

# Ask the user for the initial condition of each room.
environment = {}
```

```
for room in ["A", "B", "C"]:
    while True:
        state = input(
            f"Is room {room} Dirty or Clean? "
        ).strip().capitalize()

        if state in ["Dirty", "Clean"]:
            environment[room] = state
            break
        else:
            print("Invalid input. Please enter Dirty or Clean.")


# Function to check whether a room is dirty.
def is_dirty(room):
    return environment[room] == "Dirty"


# Function to clean a room.
def clean_room(room):
    environment[room] = "Clean"
    print(f"Room {room} has been cleaned.")


# VACUUM AGENT

class VacuumAgent:
    def __init__(self):
        # The agent starts in room A.
        self.location = "A"

    # Display a text-based grid of all rooms.
    def display_grid(self):
        print("\n--- ROOM GRID ---")

        for room in ["A", "B", "C"]:
            if room == self.location:
                agent_symbol = "[A-Agent]"
            else:
                agent_symbol = "         "

            if environment[room] == "Dirty":
```

```
                room_symbol = "[D-Dirty]"
            else:
                room_symbol = "[C-Clean]"

            print(f"Room {room}: {room_symbol} {agent_symbol}")

        print("-----------------\n")

    # Move to a randomly selected room.
    def move(self, target_room):
        self.location = target_room
        print(f"Moving to room {self.location}")

    # Decide whether to clean or move.
    def perceive_and_act(self):
        print(f"Agent is currently in room {self.location}")

        # Rule 1: Clean the current room if it is dirty.
        if is_dirty(self.location):
            clean_room(self.location)

        # Rule 2: If the current room is clean,
        # randomly choose a dirty room to visit.
        else:
            dirty_rooms = [
                room for room in environment
                if is_dirty(room)
            ]

            if dirty_rooms:
                next_room = random.choice(dirty_rooms)
                self.move(next_room)

            else:
                print("All rooms are clean! Agent is idle.")


# RUN SIMULATION

agent = VacuumAgent()

while True:
    try:
```

```
        steps = int(
            input("\nHow many steps should the agent run? ")
        )

        if steps > 0:
            break
        else:
            print("Please enter a positive number.")

    except ValueError:
        print("Invalid input. Enter a whole number.")


# Execute the simulation.
for step in range(1, steps + 1):
    print(f"\nSTEP {step} ")

    # Show the agent and room conditions before acting.
    agent.display_grid()

    # Let the agent perceive and act.
    agent.perceive_and_act()

    # Show the updated environment.
    print("Environment state:", environment)

    # Stop if all rooms are clean.
    if all(
        state == "Clean"
        for state in environment.values()
    ):
        print("\nAll rooms are clean! Agent is idle.")
        agent.display_grid()
        break

print("\nSimulation finished.")
```

Output

```
Is room A Dirty or Clean? Clean
Is room B Dirty or Clean? Dirty
Is room C Dirty or Clean? Dirty

How many steps should the agent run? 5

STEP 1

--- ROOM GRID ---
Room A: [C-Clean] [A-Agent]
Room B: [D-Dirty]
Room C: [D-Dirty]
-----------------

Agent is currently in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}

STEP 2

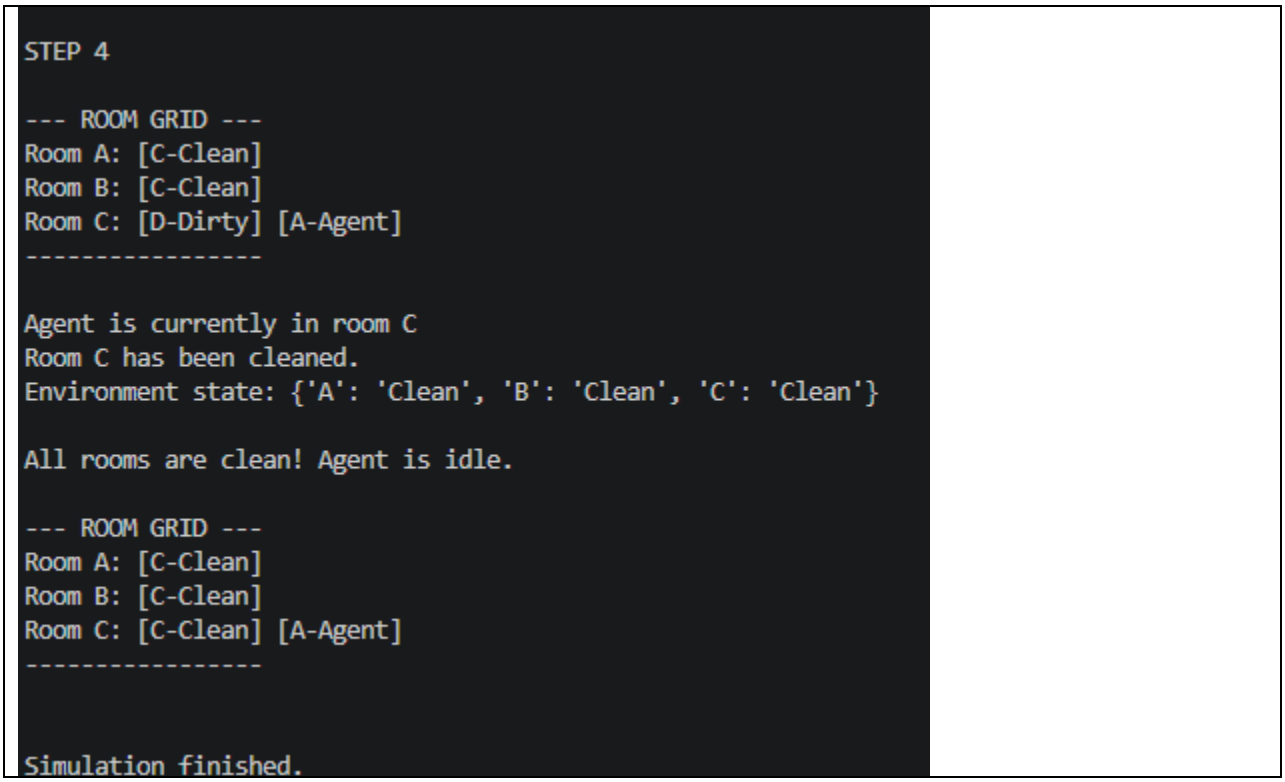
--- ROOM GRID ---
Room A: [C-Clean]
Room B: [D-Dirty] [A-Agent]
Room C: [D-Dirty]
-----------------

Agent is currently in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

STEP 3

--- ROOM GRID ---
Room A: [C-Clean]
Room B: [C-Clean] [A-Agent]
Room C: [D-Dirty]
-----------------

Agent is currently in room B
Moving to room C
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}
```



Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
   - Agent cleans the room if it's dirty, it goes to random dirty room if the current room is clean, and agent becomes idle if all the rooms are clean.
2. How does the agent decide when to clean and when to move?
   - The agent checks the room it's currently in. If it's dirty, it cleans it, if clean, move to a random dirty room.
3. What happens if both rooms are already clean at the start?
   - If all rooms are already clean from the start, the agent simple becomes idle and stay in the current room until all steps are done.
4. How would you modify the agent to handle **more than two rooms**?
   - To make the agent handle more rooms, I can simply modify the dictionary to add or remove rooms and use a loop to check their conditions, and instead of hardcoding the agent's movement

into checking both rooms, the agent can simple randomly check any other rooms included in the dictionary that is labeled as ‘dirty.’